Aerial Object Monitoring System Using LEO/MEO/GEO Satellite Constellations – Physics Engine Conceptual Design

1.0 Overview

The MLSS UAV detection system shall run on a physics engine written in C++, aiming to take advantage of its multithreading capability. This keeps the engine as lightweight as possible, which is crucial when simulating constellations of hundreds of satellites. Trajectory calculation and modelling will be offloaded to the GPU for speed. After running a simulation, a table of time-stamped (x, y, z) values detailing the positions of satellites relative to Earth will be generated. Ground stations and UAVs will have their positions preset and represented by latitude, longitude, and altitude coordinates.

[Addendum] positions of ground stations and UAVs should also be calculated in this engine and output directly, in the same format as the satellites, in which case the engine should also include unit conversion. It's also worth noting that minor positional error (~1-2 km) is probably acceptable for this engine, as the main focus is the evaluation of multilayer satellite constellations; positional hyper precision would be losing the plot. Multiple runs of configurations would iron out this error for any meaningful capacity towards evaluation. That being said, higher complexity models will still be mentioned for their potential value depending on project goals.

2.0 Tools to Consider

2.1 NASA's SPICE System

SPICE is an information system compiled by the Navigation and Ancillary Information Facility (NAIF) to assist NASA engineers in modelling and executing activities required for planetary exploration missions. SPICE datasets are referred to as "kernels" and these kernels are composed of navigation information that openly provides precision observation geometry for use to engineering communities. The SPICE system also includes a toolkit that contains a library of APIs. With these APIs, the exact positions and oblateness constants of celestial bodies can be queried from the planetary ephemerides. Additionally, SPICE's time kernels can feed precise and consistent times to the orbital engine, rectifying any potential for silent drifting errors due to time scales. Finally, SPICE can compute ground-station visibility windows and the FOV coverage compared to a point on Earth or in the Earth's atmosphere, which is an essential piece of functionality for the simulator. These APIs should be considered at the later stages of development to accurately depict the position and shape of Earth, the time conversions, and the calculations of satellite FOVs and what they detect.

The SPICE Toolkit is offered in C, but there are third party libraries that offer nearly all of the CSPICE functionality in Python. This interface is called SpiceyPy and can be pip installed. If these calculations need to be handled in the physics engine, there is also an

astrodynamics toolkit for C++ offered here: https://github.com/IO-Aerospace-software-engineering/SDK.

3.0 Implementation Steps

Step 1:

Construct a two-body propagator to model the simplest possible orbit of a satellite around the Earth.

- Represent the satellite as position and velocity vectors
- Implement an Runge-Kutta 4 (RK4) integrator for numerically solving differential equations, allowing for modelling orbital trajectories
- Output x,y,z to a CSV file for each time stamp (should probably handle timestamps in epoch time internally, displaying in UTC)

4.0 Engine Classes

Each class should be as simple as possible while staying extendable. This allows rapid prototyping and helps surface performance issues early. The classes cover classical kinematics and dynamics only. Factors such as propagation delay and electromagnetic effects are too involved for the engine and can instead be modelled as a delay or a probabilistic uncertainty elsewhere. Near-negligible forces, and forces that real satellites counteract with thrusters, are assumed to be handled by the real-life satellite system that this tool may assist, as it is responsible for countering orbital decay. The user is also responsible for all that is required to place the real-life constellation in position, since the engine does not simulate launches.

Unless stated otherwise, each class needs a constructor and any utility methods deemed necessary.

Units and time: distances in km, speeds in km/s, angles in radians internally (degrees only for the UI and front-end viewing). Internally, time is treated as continuous, in seconds since the start of the run, because the correct position can be accessed directly at any time t. Results are written at a stored time step (the storage interval mentioned in 1.4 of the conceptual design doc), and user friendly coordinated universal time (UTC) is derived only at output. The engine calculates in a non-rotating frame or an Earth Centered Inertial (ECI) frame and converts to Earth-centered Earth-fixed (ECEF) only when writing output (more readable).

4.1 Satellite Body

The Satellite Body is a simple "particle" class, as seen in other orbital gravity simulators. It has no mass variable, since gravitational acceleration is independent of the mass of the orbiting body and other inertial factors are irrelevant in non-decaying orbits. Instead of stepping forward from the previous position, each satellite is described by six orbital values set at the start (with the node angle drifting at the J2 rate), and its position at any time t is calculated directly from them using Kepler's equation. J2 is included as a fixed drift rate of the orbit's rotation around Earth's axis.

Instance variables:
Orbital elements for Kepler:

- Semi-major axis (a): the size of the orbit. It is half the longest width of the ellipse, in km from Earth's center. For a circular orbit it is just the orbital radius (Earth's radius + altitude). It also fixes the period and speed.
- Eccentricity (e): the shape of the orbit. 0 is a perfect circle, values approaching 1 are long, stretched ellipses. Always 0 for now, since every orbit is circular.
- Inclination (i): the tilt of the orbital plane relative to Earth's equator. 0° orbits along the equator, 90° passes over the poles.
- Right ascension of the ascending node (Ω): how the tilted plane is rotated around Earth's axis. It is the angle, measured around the equator, to the point where the satellite crosses the equator going north. This is the value J2 slowly drifts.
- Argument of perigee (ω): where the closest point of the orbit sits within the plane, measured as an angle from the ascending node. For a circular orbit it has no real meaning and is always set to 0.
- True anomaly (v): where the satellite is along its orbit at the start, measured as an angle from the closest point. This sets where it starts, and it is the only one of the six that changes quickly on its own.
  Non-Kepler:
- J2 drift rate (float): rate at which the orbit rotates around Earth's axis, in radians per second. Calculated once from the elements.

https://www.sternula.com/space-curiosity-keplerian-elements/

We should probably include a diagram illustrating these angles and parameters

Methods:

- GetState(float t) returns position and velocity (Vector3 each) at time t. This is the only method needed each step, and any timestamp can be calculated independently of the others.
- Constructor: converts the config into a position and velocity in the non-rotating frame, then into the six elements, and calculates the J2 drift rate. Also runs the early rejection checks (orbit dipping below about 200 km altitude or intersecting Earth).

Constructor inputs:

- Longitude, latitude, altitude: where the satellite starts.
- Heading: compass direction of travel at the start (0° = north, 90° = east). The velocity is horizontal, so it always points along this heading.
  Every satellite starts at t = 0 for now.

Input modes (both feed the same constructor):

- Single satellite: longitude, latitude, altitude and heading.
- Preset batch: height, tilt, number of satellites and number of orbital planes, which generates the satellites directly.

Note: velocity is not an input. It is always the circular orbit speed for the altitude, v = root(GM/r), so it cannot be wrong, and eccentricity and argument of perigee are always 0 for now.

4.2 Earth

Earth is a point mass at the origin that never moves. Its rotation is used only for the frame conversion on output (the front end uses the same rotation to draw it), so it only needs to hold constants and can be a singleton. Satellite Bodies do not need to hold a reference to it if defined as a singleton.

Constants:

- Position: (0, 0, 0)
- Mu: the gravitational parameter GM, about 398,600.44 km^3/s^2
- Equatorial radius, J2 coefficient, and rotation rate: used by the J2 drift and the frame conversion
- Flattening: used only for latitude/longitude/altitude conversion in 4.5
  Oblateness is handled through J2 alone, so a separate oblate spheroid model is not

needed. But this models the oblateness for positional importance of ground stations and aircraft.

4.3 Aircraft

The aircraft is unrelated to gravitational physics. The engine does not simulate it. It replays a predetermined path and outputs its position in the same format as satellites.

Instance variables:

- Path (List of Vector4: longitude, latitude, altitude, time): real flight logs already come with a time for each point. If the source data only gives positions and a speed, the times can be calculated at load from segment length / speed.
- Active window (start and end time): outside this window the aircraft does nothing.
- Signature: a numerical value representing how detectable the aircraft is. The engine does not use this; it is kept here so the sensor model can reference one aircraft record.

Methods:

- GetPosition(float t finds the position at time t based on the closest recorded points before and after it. The path is converted to engine coordinates once when loaded. Repeating flights can be handled later by a backend control that advances the time stamps uniformly.

4.4 Ground Station

Similar to the Aircraft, but stationary, so only a position is needed.

Instance variables:

- Position: converted at load, since it never changes in the output frame.
- Class/identifier: Not necessarily required to be a variable, as long as other systems know it is of class Ground Station.

Methods:

- GetClosest(Vector3 point) (static): returns the ground station closest to a given point.

4.5 Unit Converter

A static helper class for all conversions the engine needs. Ground stations and aircraft are given as longitude, latitude and altitude, so they are converted to Earth-fixed x, y, z (km) at load to match the engines output format (time stamped x, y, z relative to Earth). It also converts source data units (depending on what the format is for disclosed flight paths) at load, the non-rotating frame to the Earth-fixed frame on output, orbital elements to position and velocity, position and velocity to orbital elements (used by the Satellite Body constructor), and heading and circular orbit speed to a velocity vector.

4.6 Alternate Design: Full Physics Model

This is the original concept, but it is more computationally involved, with limited benefit for this project's needs. It is kept as the fallback in the model options, and as a way to potentially validate the analytic model. It is also required if drag or third-body forces are ever added however this depends on the complexity required for the other components of this project. Below are the changes that would have to be made to the analytical model.

Satellite Body (numerical) instance variables:

- Position (Vector3)
- Velocity (Vector3)

Satellite Body (numerical) methods:

- Step(float dt): advances the satellite by one time step using an RK4 integrator. Only every Nth step, matching the stored time step, is written to the output.
- Acceleration: J2 is included as a small extra acceleration term.

Earth (added method):

- GetGravityAcceleration(Vector3 satellitePosition) returns a Vector3. It calculates with the formula $-(GM / r^3) *$ the position vector.

Trade-offs versus the analytic design: every timestamp depends on the one before it, so the time step must be small, and errors accumulate over long runs. It is also harder to run safely at lower GPU precision. Satellites are still independent of each other, so they can be calculated in parallel.

References:

https://github.com/ButteredFire/Astrocelerate/tree/main

Really good example of what we wanna make but it's probably a lot more advanced than we need

https://naif.jpl.nasa.gov/pub/naif/toolkit_docs/Tutorials/pdf/individual_docs/03_spice_overview.pdf

https://naif.jpl.nasa.gov/naif/spiceconcept.html

https://naif.jpl.nasa.gov/naif/toolkit.html

SPICE, NASA's observation geometry system (kernels are also used by astrocelerate)

https://en.wikipedia.org/wiki/Two-body_problem

Contains useful equations relating to the motion, energy, and central forces

https://www.youtube.com/watch?v=HOWJp8NV5xU

https://www.youtube.com/watch?v=vNoFdtcPFdk

Runge Kutta 4 Integrator Videos (conceptual and mathematical explanation + code)

Orbits:

https://satkit.dev/tutorials/Keplerian%20Elements/ - mainly relevant

https://en.wikipedia.org/wiki/Orbital_elements - a bit more fields not required (for more complex orbits)

https://en.wikipedia.org/wiki/Kepler%27s_equation - full equation(s) for the given orbital elements

https://en.wikipedia.org/wiki/Nodal_precession - J2 effect on Keplerian orbit

https://astronomy.stackexchange.com/questions/18761/j2-perturbations-and-orbits - same

https://ai-solutions.com/_freeflyeruniversityguide/j2_perturbation.htm -practical source, too specific with it accounting for sun though